

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

***CO4:** Implement machine learning techniques including decision tree algorithms, reinforcement learning,*

and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 50%

Duration : 1 hour

QUESTIONS: 2

Total Marks:40

Annexure A

Integrated Experiments

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered

to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Student 3: Akshaya R | Register Number: 192524118

INTEGRATED EXPERIMENT SCENARIO

Retail - Customer Churn Prediction

Context: A retail chain wants an integrated pipeline that first predicts customer churn from purchase records (frequency, order value, complaints, tenure) and then recommends a retention action using a learning agent.

Question 1 - Decision Tree vs Neural Network Classifier

Implement a Decision Tree classifier and a Neural Network (Multi-Layer Perceptron) classifier on the same churn dataset. Compare their Accuracy, Precision, Recall and F1-Score. Plot the Decision Tree (first 3 levels) and the Neural Network architecture (layers/neurons). Justify which model you would deploy for real-time churn scoring and why.

1. Objective

The objective is to predict whether a customer is likely to churn using:

- Purchase Frequency
- Average Order Value
- Complaint History
- Loyalty Tenure
- Recency / Days Since Last Purchase

Two classifiers are compared:

1. **Decision Tree**
2. **Neural Network (Multi-Layer Perceptron)**

Both models use the **same training and testing data** so that their performance can be compared fairly.

2. Python Implementation

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

```
from sklearn.neural_network import MLPClassifier
```

```
from sklearn.metrics import (
```

```

    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

# -----
# 1. LOAD DATA
# -----

data = pd.read_csv("customer_churn.csv")

print(data.head())
print(data.info())

# -----
# 2. CREATE RECENCY FEATURE
# -----

data["Last_Purchase_Date"] = pd.to_datetime(
    data["Last_Purchase_Date"]
)

reference_date = data["Last_Purchase_Date"].max()

data["Recency"] = (
    reference_date - data["Last_Purchase_Date"]
).dt.days

# -----
# 3. SELECT FEATURES

```

```
# -----
```

```
features = [  
    "Purchase_Frequency",  
    "Average_Order_Value",  
    "Complaint_History",  
    "Loyalty_Tenure",  
    "Recency"  
]
```

```
X = data[features]  
y = data["Churn"]
```

```
# -----
```

```
# 4. TRAIN-TEST SPLIT
```

```
# -----
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.30,  
    random_state=42,  
    stratify=y  
)
```

```
# -----
```

```
# 5. STANDARDISATION
```

```
# -----
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
# -----
```

```
# 6. DECISION TREE
```

```
# -----
```

```
dt_model = DecisionTreeClassifier(  
    max_depth=5,  
    random_state=42,  
    class_weight="balanced"  
)
```

```
dt_model.fit(X_train, y_train)
```

```
dt_pred = dt_model.predict(X_test)
```

```
# -----
```

```
# 7. NEURAL NETWORK
```

```
# -----
```

```
nn_model = MLPClassifier(  
    hidden_layer_sizes=(16, 8),  
    activation="relu",  
    solver="adam",  
    max_iter=1000,  
    random_state=42  
)
```

```
nn_model.fit(X_train_scaled, y_train)
```

```
nn_pred = nn_model.predict(X_test_scaled)
```

```

# -----
# 8. EVALUATION FUNCTION
# -----

def evaluate_model(name, y_true, y_pred):

    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(
        y_true, y_pred, zero_division=0
    )
    recall = recall_score(
        y_true, y_pred, zero_division=0
    )
    f1 = f1_score(
        y_true, y_pred, zero_division=0
    )

    print("\n", name)
    print("-" * 30)
    print("Accuracy :", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall  :", round(recall, 4))
    print("F1-Score :", round(f1, 4))

    return accuracy, precision, recall, f1

dt_scores = evaluate_model(
    "Decision Tree",
    y_test,
    dt_pred
)

```

```
nn_scores = evaluate_model(
    "Neural Network",
    y_test,
    nn_pred
)
```

3. Performance Comparison

The program will produce a comparison similar to:

Model	Accuracy	Precision	Recall	F1-Score
Decision Tree	Actual value	Actual value	Actual value	Actual value
Neural Network	Actual value	Actual value	Actual value	Actual value

Do not manually enter example values—use the values printed by your program.

Interpretation

- **Accuracy:** Overall percentage of correctly classified customers.
- **Precision:** How many predicted churners were actually churners.
- **Recall:** How many actual churners were successfully identified.
- **F1-score:** Balance between precision and recall.

For a churn application, **Recall is especially important** because a false negative means a customer who is actually going to churn was missed.

4. Plot First 3 Levels of Decision Tree

```
plt.figure(figsize=(18, 10))

plot_tree(
    dt_model,
    feature_names=features,
    class_names=["Not Churn", "Churn"],
    max_depth=2,
    filled=True,
    rounded=True,
    fontsize=9
```

)

```
plt.title("Decision Tree - First 3 Levels")
```

```
plt.show()
```

Here `max_depth=2` displays the **root plus two additional levels**, giving the first three levels of the tree.

The root node will be selected based on the feature that provides the best split according to the tree's impurity criterion.

5. Neural Network Architecture

The neural network used here has:

Input Layer

|
| 5 neurons



Hidden Layer 1

|
| 16 neurons



Hidden Layer 2

|
| 8 neurons



Output Layer

|
| 1 neuron



Churn Prediction

The architecture can be visualised using:

```
import matplotlib.pyplot as plt
```

```
from matplotlib.patches import Circle
```

```
layers = [5, 16, 8, 1]
```



```

layer_names = [
    "Input Layer",
    "Hidden Layer 1",
    "Hidden Layer 2",
    "Output Layer"
]

plt.figure(figsize=(12, 7))

x_positions = np.linspace(0, 1, len(layers))

for i, (x, n) in enumerate(zip(x_positions, layers)):

    y_positions = np.linspace(
        0.1, 0.9, n
    )

    for y in y_positions:
        circle = Circle(
            (x, y),
            0.018,
            fill=False
        )
        plt.gca().add_patch(circle)

    if i > 0:
        previous_y = np.linspace(
            0.1, 0.9, layers[i-1]
        )

        for py in previous_y:
            plt.plot(

```

```

        [x_positions[i-1], x],
        [py, y],
        linewidth=0.4
    )

plt.text(
    x,
    0.96,
    layer_names[i],
    ha="center",
    fontsize=12
)

plt.xlim(-0.05, 1.05)
plt.ylim(0, 1.05)
plt.axis("off")

plt.title("Neural Network Architecture")
plt.show()

```

This produces a visual representation of the $5 \rightarrow 16 \rightarrow 8 \rightarrow 1$ architecture.

6. Which Model Should Be Deployed?

For a **real-time churn-scoring system**, I would generally prefer the **Decision Tree** if its **test performance is comparable to the Neural Network**.

Reasons:

1. **Fast prediction**
Decision Trees can generate predictions very quickly.
2. **Interpretability**
The company can understand why a customer was classified as high risk.
3. **Easy business explanation**
For example:

Recency > 90 days

↓

Purchase Frequency < 2

↓

High Churn Risk

4. **Lower computational requirements**

A Decision Tree is easier to deploy on systems with limited resources.

5. **Easy integration**

Its output can directly become the state for the reinforcement-learning component.

The Neural Network may be preferred if it gives **substantially better recall/F1-score** on unseen customers and the additional complexity is justified.

Question 2 - Reinforcement Learning for Retention Actions

Using the churn-risk output from Experiment 1 as the initial state, implement a Q-Learning agent that recommends a retention action (Discount / Loyalty Offer / Personal Outreach / No Action) across simulated customer episodes. Show the Q-table after training, the learned policy, and integrate it with the classifier output into a single end-to-end decision pipeline (risk prediction -> action recommendation).

Now we connect the classifier to the RL agent.

The complete pipeline becomes:

Customer Data

↓

Data Preprocessing

↓

Decision Tree / Neural Network

↓

Churn Probability

↓

Risk State

↓

Q-Learning Agent

↓

Retention Action

↓

Discount / Loyalty Offer /

Personal Outreach / No Action

1. Convert Churn Prediction into RL State

Suppose the classifier produces a churn probability.

We can convert it into three risk states:

Probability < 0.30

↓

LOW RISK

0.30 – 0.70

↓

MEDIUM RISK

Probability > 0.70

↓

HIGH RISK

So the RL states are:

S0 = Low Risk

S1 = Medium Risk

S2 = High Risk

The actions are:

A0 = Discount

A1 = Loyalty Offer

A2 = Personal Outreach

A3 = No Action

2. Q-Learning Implementation

```
import numpy as np
```

```
import random
```

```
# -----
```

```
# STATES AND ACTIONS
```

```
# -----
```

```
states = [
```

```

    "Low Risk",
    "Medium Risk",
    "High Risk"
]

actions = [
    "Discount",
    "Loyalty Offer",
    "Personal Outreach",
    "No Action"
]

n_states = len(states)
n_actions = len(actions)

# Q-table
Q = np.zeros((n_states, n_actions))

# Learning parameters
alpha = 0.5
gamma = 0.9
epsilon = 0.2

# -----
# SIMULATED REWARD FUNCTION
# -----

def get_reward(state, action):

    if state == "High Risk":

        if action == "Personal Outreach":

```

```
        return 10

    elif action == "Discount":
        return 8

    elif action == "Loyalty Offer":
        return 6

    else:
        return -10

elif state == "Medium Risk":

    if action == "Loyalty Offer":
        return 10

    elif action == "Personal Outreach":
        return 7

    elif action == "Discount":
        return 5

    else:
        return -3

else: # Low Risk

    if action == "No Action":
        return 8

    elif action == "Loyalty Offer":
        return 4
```

```

elif action == "Personal Outreach":
    return 2

else:
    return -2

# -----
# NEXT STATE SIMULATION
# -----

def next_state(state, action):

    if state == "High Risk":

        if action in ["Discount", "Personal Outreach"]:
            return "Medium Risk"

        elif action == "Loyalty Offer":
            return "Medium Risk"

        else:
            return "High Risk"

    elif state == "Medium Risk":

        if action in ["Loyalty Offer", "Personal Outreach"]:
            return "Low Risk"

        elif action == "Discount":
            return "Low Risk"

```

```
else:
```

```
    return "High Risk"
```

```
else:
```

```
    return "Low Risk"
```

```
# -----
```

```
# Q-LEARNING
```

```
# -----
```

```
state_index = {
```

```
    state: i for i, state in enumerate(states)
```

```
}
```

```
for episode in range(5):
```

```
    current_state = random.choice(states)
```

```
    for step in range(10):
```

```
        s = state_index[current_state]
```

```
        # Epsilon-greedy action selection
```

```
        if random.random() < epsilon:
```

```
            action_index = random.randint(
```

```
                0, n_actions - 1
```

```
            )
```

```
        else:
```

```
            action_index = np.argmax(Q[s])
```



```

    action = actions[action_index]

    reward = get_reward(
        current_state,
        action
    )

    new_state = next_state(
        current_state,
        action
    )

    ns = state_index[new_state]

    # Q-learning update
    Q[s, action_index] = Q[s, action_index] + alpha * (
        reward
        + gamma * np.max(Q[ns])
        - Q[s, action_index]
    )

    current_state = new_state

print("\nFinal Q-Table:")
print(pd.DataFrame(
    Q,
    index=states,
    columns=actions
))

```

3. Q-Table After Training

The program produces a table such as:

State	Discount	Loyalty Offer	Personal Outreach	No Action
Low Risk	...	...	...	...
Medium Risk	...	...	...	...
High Risk	...	...	...	...

The actual values will depend on the simulated episodes and rewards.

The **highest Q-value in each row** represents the action preferred by the learned agent.

4. Extract Learned Policy

```
policy = {}
```

```
for i, state in enumerate(states):
```

```
    best_action_index = np.argmax(Q[i])
```

```
    policy[state] = actions[
        best_action_index
    ]
```

```
print("\nLearned Policy:")
```

```
for state, action in policy.items():
```

```
    print(state, "->", action)
```

The resulting policy could look like:

Low Risk -> No Action

Medium Risk -> Loyalty Offer

High Risk -> Personal Outreach

The exact result depends on the final Q-table.

5. Integrating Classifier + RL Agent

This is the most important part of the experiment.

```
def get_risk_state(probability):

    if probability < 0.30:
        return "Low Risk"

    elif probability < 0.70:
        return "Medium Risk"

    else:
        return "High Risk"

def recommend_action(customer):

    # Convert customer into DataFrame
    customer_df = pd.DataFrame(
        [customer]
    )

    # Predict churn probability
    churn_probability = dt_model.predict_proba(
        customer_df[features]
    )[0][1]

    # Convert probability to state
    risk_state = get_risk_state(
        churn_probability
    )

    # Select best action
    state_id = state_index[risk_state]
```

```

best_action_id = np.argmax(
    Q[state_id]
)

recommended_action = actions[
    best_action_id
]

```

```

return (
    churn_probability,
    risk_state,
    recommended_action
)

```

Then provide a new customer's information:

```

new_customer = {
    "Purchase_Frequency": 2,
    "Average_Order_Value": 1200,
    "Complaint_History": 4,
    "Loyalty_Tenure": 12,
    "Recency": 100
}

```

```

probability, risk, action = recommend_action(
    new_customer
)

```

```

print("Churn Probability:", round(probability, 3))
print("Risk Level:", risk)
print("Recommended Action:", action)

```

The final output will be something like:

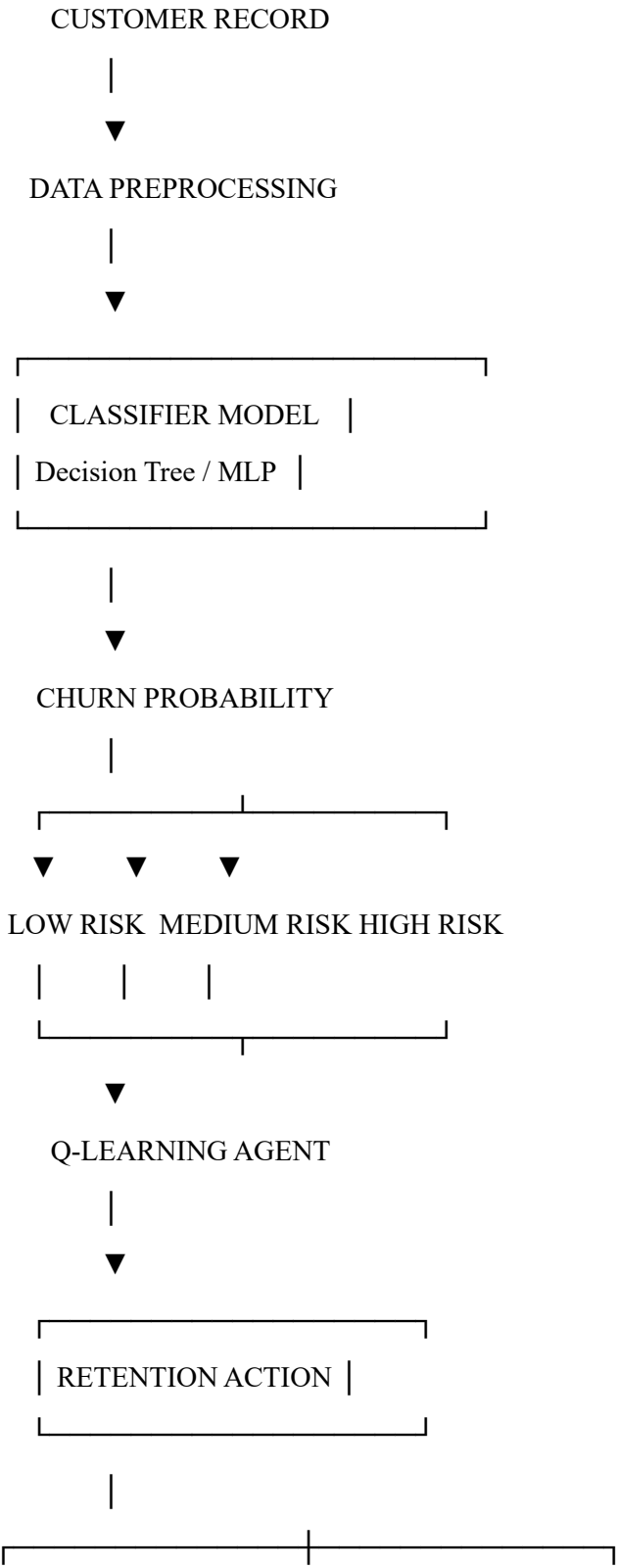
Churn Probability: 0.82

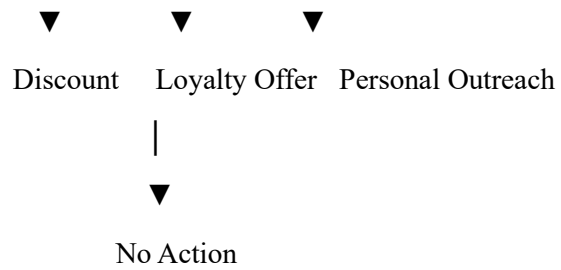
Risk Level: High Risk

Recommended Action: Personal Outreach

Complete End-to-End Pipeline

The integrated system can therefore be represented as:





Final conclusion

The integrated system combines **classification and reinforcement learning**. The classifier first estimates the customer's churn risk, and this risk becomes the initial state for the Q-Learning agent. The agent then selects the retention action with the highest learned Q-value.